An Introduction to Emacs Lisp

We have covered a lot of ground in our quest to learn the intricacies of Emacs and we trust that you have found your long journey fulfilling. This is the last article in the series.

The GNU Emacs editor is written using Emacs Lisp and the C programming language. You can use GNU Emacs as an editor without knowing Emacs Lisp, but being familiar with it will help you customise it or extend it according to your needs. Emacs Lisp is a dialect of the Lisp programming language and is inspired by Maclisp and Common Lisp. The source files end with the file name extension *.el* and the byte-compiled files end with the *.elc* file name extension. You can also write scripts using Emacs Lisp and execute them as a batch operation. The code can thus be executed from the command line or from an executable file. Byte-compiling the source files can help you speed up the execution.

Comments

All comments begin with a semi-colon. An Emacs Lisp file usually has the following sections in the code, specified with three semi-colons. These always start in the left margin as shown below:

```
;;; Module --- Summary

;;; Commentary:

;;; Code:

;;; Module ends here
```

All comments outside functions begin with two semi-colons. The contents of the scratch buffer, shown below, are an example:

```
;; This buffer is for notes you don't want to save, and for
Lisp evaluation.
;; If you want to create a file, visit that file with C-x C-f,
;; then enter the text in that file's own buffer.
```

Comments inside functions use a single semi-colon, and if they span multiple lines, they must be neatly aligned. For example:

```
...
(let* ((sexp (read (current-buffer)))   ; using 'read' here
                                        ; easier than regexp
                                        ; matching, esp. with
                                        ; different forms of
                                        ; MONTH
 ...
 ))
```

Literals

The basic data types are available in Emacs Lisp. Numbers can be represented by integers or floats. Integers can have their sign representation before the digit (+1, -2). Floating point numbers can be represented using a decimal point (3.1415) or with an exponent (314.15e-2). A character (S) is represented by its integer code (83), while a string is a list of characters enclosed within double quotes ("A string").

A symbol is an object with a name. A keyword symbol is one that starts with a colon (:). A vector is an array and can contain different types ([1 "two" :three]). The Boolean values for true and false are 't' and 'nil' respectively. A *cons* cell is an object with two slots. The first slot is called the CAR (Contents of the Address part of the Register number) and the second slot is called the CDR (Contents of the Decrement part of the Register number). A list is a series of linked cons cells. For example, in the list '(A B)', the CAR is A and the CDR is B.

Sexp

Emacs Lisp uses prefix notation, which consists of an operation followed by operands (arguments). All programs are represented as symbolic expressions (sexp). For example, the '+' operation is applied to its arguments '1' and '2' in the following sexp:

```
(+ 1 2)
```

If you copy the above code in an Emacs buffer, you can evaluate the same by placing the cursor at the end of the expression and using the C-x C-e shortcut. The output '3' will